

Truthiness, Logic & Comparisons

Section 3 — JavaScript Fundamentals

JavaScript-এ আমরা প্রায় প্রতিদিনই এই চারটি concept ব্যবহার করি:

- **Truthy & Falsy Values**
- **Logical Operators**
- **Short-Circuit Evaluation**
- **Comparison Operators**

React, form validation, API response handling, authentication, conditional rendering—সব জায়গাতেই এগুলো গুরুত্বপূর্ণ।

এই chapter-এর সবচেয়ে important idea হলো:

JavaScript অনেক জায়গায় *boolean* না থাকলেও কোনো *value*-কে `true` বা `false` হিসেবে *evaluate* করতে পারে।

1. Boolean Context

প্রথমে বুঝতে হবে **Boolean Context** কী।

ধরো:

```
const age = 25;

if (age) {
  console.log("Age exists");
}
```

এখানে `age` সরাসরি *boolean* নয়।

```
age → 25
```

তবুও `if` condition-এর মধ্যে JavaScript `25`-কে *truthy* হিসেবে *evaluate* করেছে।

তাই:

```
25 → true
```

এরকম context-কে *Boolean Context* বলা যায়।

Common Boolean contexts:

```
if (value) {  
  }  
  
while (value) {  
  }  
  
!value  
  
value && something  
  
value || something  
  
value ? a : b
```

2. Truthy Values

যে value Boolean context-এ `true` হিসেবে behave করে, সেটি **truthy value**।

Example:

```
if ("hello") {  
  console.log("Truthy");  
}
```

Output:

```
Truthy
```

কারণ non-empty string truthy।

2.1 Common Truthy Values

Examples:

```
true
1
-1
"hello"
"0"
[]
{}
function () {}
```

খেয়াল করো:

```
"0"
```

একটি **string**, তাই এটি truthy।

3. Falsy Values

যে value Boolean context-এ `false` হিসেবে behave করে, সেটি **falsy value**।

JavaScript-এ falsy values-এর তালিকা খুব গুরুত্বপূর্ণ।

Common falsy values:

```
false
0
-0
0n
""
null
undefined
NaN
```

Modern JavaScript-এ `document.all`-এর একটি বিশেষ legacy falsy behavior আছে, কিন্তু everyday application code-এ সেটি practically relevant নয়।

4. Falsy Values মনে রাখার সহজ উপায়

এইগুলো মুখস্থ রাখো:

```
false
0
-0
0n
""
null
undefined
NaN
```

আর সবচেয়ে গুরুত্বপূর্ণ:

Empty string `""` *falsy*, কিন্তু `" "` *truthy*!

কারণ `" "`-এ একটি space character আছে।

5. `Boolean()` দিয়ে Truthy/Falsy Check

কোনো value truthy নাকি falsy manually দেখতে:

```
Boolean(value)
```

Example:

```
console.log(Boolean(1));
// true

console.log(Boolean(0));
// false

console.log(Boolean("hello"));
// true

console.log(Boolean(""));
// false
```

6. `!!` দিয়ে Boolean Conversion

Double NOT operator দিয়েও Boolean conversion করা যায়:

```
!!value
```

Example:

```
console.log(!!"hello");  
// true  
  
console.log(!! "");  
// false  
  
console.log(!!0);  
// false  
  
console.log(!!100);  
// true
```

Mental model:

```
!value  
  ↓  
boolean-এ convert + reverse  
  
!!value  
  ↓  
boolean-এ convert
```

তাই:

```
!!value
```

অনেক codebase-এ truthiness-এর explicit boolean result পাওয়ার জন্য দেখা যায়।

7. Empty String vs Space

এটা খুব common interview/output question।

```
Boolean("");
```

Output:

```
false
```

কিন্তু:

```
Boolean(" ");
```

Output:

```
true
```

কারণ:

```
"" → কোনো character নেই  
" " → একটি space character আছে
```

8. Empty Array Is Truthy

অনেক beginner ভাবে empty array falsy।

কিন্তু:

```
Boolean([]);
```

Output:

```
true
```

অর্থাৎ:

```
if ([]) {  
  console.log("Runs");  
}
```

এটি run করবে।

9. Empty Object Is Also Truthy

```
Boolean({});
```

Output:

```
true
```

তাই:

```
if ({}) {  
    console.log("Runs");  
}
```

এটিও run করবে।

Mental model:

```
[] → truthy  
{ } → truthy
```

10. Why Are `[]` and `{ }` Truthy?

কারণ JavaScript-এ object values সাধারণত truthy।

Array হলো object-এর একটি form এবং plain object-ও object।

তাই:

```
Boolean([]);  
// true  
  
Boolean({});  
// true
```

যদি array empty কিনা check করতে চাও:

```
if (items.length === 0) {  
      
}
```

এটা ব্যবহার করো।

শুধু:

```
if (items) {  
  }
```

দিয়ে empty array detect করা যাবে না।

11. Logical Operators

JavaScript-এর তিনটি primary logical operator:

```
&& → AND  
|| → OR  
! → NOT
```

এগুলো boolean logic-এর পাশাপাশি JavaScript-এ **value selection**-এর জন্যও ব্যবহৃত হয়।

এটা খুব important!

12. AND Operator &&

Basic logic:

```
true && true → true  
true && false → false  
false && true → false  
false && false → false
```

অর্থাত্:

দুই পাশই true হলে AND result true।

13. Basic `&&` Example

```
const age = 25;
const hasTicket = true;

if (age >= 18 && hasTicket) {
  console.log("You can enter");
}
```

এখানে দুইটি condition true হতে হবে।

14. `&&` শুধু Boolean Return করে না

JavaScript-এর একটি গুরুত্বপূর্ণ behavior:

```
console.log("Hello" && "World");
```

Output:

```
World
```

এখানে `&&` boolean `true` return করেনি।

এটি operand-এর actual value return করেছে।

15. `&&` এর Mental Model

JavaScript-এ:

```
A && B
```

এর mental model:

A falsy হলে A return করো, না হলে B return করো।

Example:

```
console.log(0 && "Hello");
```

Output:

```
0
```

কারণ `0` falsy।

আর:

```
console.log("Hello" && "World");
```

Output:

```
World
```

কারণ `"Hello"` truthy, তাই second operand evaluate করা হয়েছে।

16. `&&` Table

ধরো:

```
A && B
```

A	B	Result
truthy	truthy	B
truthy	falsy	B
falsy	truthy	A
falsy	falsy	A

এখানে result actual operand value হতে পারে।

17. `||` OR Operator

Basic boolean logic:

```
true || true → true
true || false → true
false || true → true
false || false → false
```

অর্থাৎ:

অন্তত একটি true হলে OR result true।

18. Value Return করে

এটিও শুধু boolean return করে না।

```
console.log("Hello" || "World");
```

Output:

```
Hello
```

কারণ first value truthy।

19. Mental Model

```
A || B
```

এর mental model:

A truthy হলে A return করো, না হলে B return করো।

Example:

```
console.log("Hello" || "World");
```

Output:

```
Hello
```

আর:

```
console.log("" || "Guest");
```

Output:

```
Guest
```

কারণ `""` falsy।

20. `||` Table

A	B	Result
truthy	truthy	A
truthy	falsy	A
falsy	truthy	B
falsy	falsy	B

21. NOT Operator `!`

`!` একটি value-এর truthiness উল্টে দেয়।

```
console.log(!true);  
// false  
  
console.log(!false);  
// true
```

21.1 Truthy Value

```
console.log(!"hello");
```

Output:

```
false
```

কারণ:

```
"hello" → truthy  
!truthy → false
```

21.2 Falsy Value

```
console.log(!0);
```

Output:

```
true
```

কারণ:

```
0 → falsy  
!falsy → true
```

22. Double NOT

```
console.log(!!"hello");  
// true  
  
console.log(!!0);  
// false
```

প্রথম  truthiness reverse করে।

দ্বিতীয়  আবার reverse করে।

ফলে final result actual boolean হয়।

23. Short-Circuit Evaluation

এখন আসি খুব important concept-এ।

Short-circuit evaluation হলো এমন behavior যেখানে JavaScript logical expression-এর result determine হয়ে গেলে পরের অংশ evaluate না-ও করতে পারে।

এটি:

```
performance
+
conditional execution
+
default values
```

এর জন্য useful।

24. **&&** Short-Circuit

ধরো:

```
false && console.log("Hello");
```

false দেখেই JavaScript জানে:

```
false && anything
= false
```

তাই second অংশ execute করবে না।

Output:

```
কিছুই print হবে না
```

25. Real-world **&&** Example

React-এ এটি খুব common:

```
{isLoggedIn && <Dashboard />}
```

যদি:

```
isLoggedIn = true
```

তাহলে:

```
<Dashboard />
```

render হবে।

আর:

```
isLoggedIn = false
```

হলে right side evaluate/render হবে না।

26. Short-Circuit

```
true || console.log("Hello");
```

JavaScript জানে:

```
true || anything  
= true
```

তাই right side execute হবে না।

27. Default Value with

খুব common pattern:

```
const userName = inputName || "Guest";
```

যদি:

```
inputName = "Shariar"
```

তাহলে:

```
userName → "Shariar"
```

আর:

```
inputName = "";
```

হলে:

```
userName → "Guest"
```

কারণ empty string falsy।

28. || এর Problem

|| যেকোনো falsy value-কে fallback trigger হিসেবে ধরে।

ধরো:

```
const quantity = 0;  
  
const result = quantity || 1;
```

Result:

```
1
```

কিন্তু হয়তো 0 একটি valid value ছিল।

এক্ষেত্রে || ভুল হতে পারে।

29. Nullish Coalescing ??

যখন শুধু null বা undefined হলে fallback দিতে চাও, তখন:

??

ব্যবহার করা ভালো।

Example:

```
const quantity = 0;

const result = quantity ?? 1;

console.log(result);
```

Output:

0

কারণ 0 nullish নয়।

30. || vs ??

Value	value "Default"	value ?? "Default"
0	"Default"	0
""	"Default"	""
false	"Default"	false
null	"Default"	"Default"
undefined	"Default"	"Default"

এই distinction frontend development-এ খুব useful।

31. Important ?? Rule

এই ধরনের expression:

a ?? b

এবং:

```
a || b
```

এক জিনিস নয়।

?? শুধু:

```
null  
undefined
```

এর জন্য fallback দেয়া

32. Logical Operator Precedence

একাধিক logical operator থাকলে precedence গুরুত্বপূর্ণ।

Example:

```
true || false && false
```

এখানে `&&` আগে evaluate হবে।

অর্থাৎ:

```
true || (false && false)
```

তারপর:

```
true || false
```

Result:

```
true
```

Confusing expression হলে parentheses ব্যবহার করো:

```
true || (false && false)
```

এতে intent clear হয়।

33. Comparison Operators

এখন আসি comparison-এ।

Common comparison operators:

```
<    → less than
>    → greater than
<=   → less than or equal
>=   → greater than or equal
```

এগুলো সাধারণত boolean result দেয়:

```
true
false
```

34. Less Than

```
console.log(5 < 10);
```

Output:

```
true
```

কারণ 5 ছোট 10-এর চেয়ে।

35. Greater Than

```
console.log(10 > 5);
```

Output:

```
true
```

36. Less Than or Equal <=

```
console.log(5 <= 5);
```

Output:

```
true
```

কারণ 5 হলো 5-এর সমান।

আর:

```
console.log(4 <= 5);
```

এটিও:

```
true
```

37. Greater Than or Equal >=

```
console.log(5 >= 5);
```

Output:

```
true
```

আর:

```
console.log(6 >= 5);
```

এটিও:

```
true
```

38. Comparison with Numbers

```
console.log(10 > 5);  
// true  
  
console.log(10 < 5);  
// false  
  
console.log(10 >= 10);  
// true  
  
console.log(10 <= 9);  
// false
```

39. Comparison with Strings

String comparison JavaScript-এ lexical order-এর ভিত্তিতে হয়।

Example:

```
console.log("apple" < "banana");
```

Output:

```
true
```

কারণ lexical ordering-এ "apple" আগে আসে।

40. String Comparison Character by Character

```
console.log("cat" < "dog");
```

c এবং d compare হয়।

```
c < d
```

তাই result:

```
true
```

41. Case Sensitivity in String Comparison

```
console.log("A" < "a");
```

এখানে Unicode code point ordering-এর কারণে result:

```
true
```

কারণ uppercase **A** এবং lowercase **a**-এর code point আলাদা।

String comparison করতে গেলে case এবং Unicode ordering সম্পর্কে aware থাকা দরকার।

42. Numeric String Comparison

এখানে coercion গুরুত্বপূর্ণ।

```
console.log("10" > 5);
```

Output:

```
true
```

এখানে numeric comparison-এর জন্য string numeric value হিসেবে coerce হতে পারে।

আর:

```
console.log("2" > "10");
```

এখানে দুইটিই string হওয়ায় lexical string comparison হতে পারে।

Result:

```
true
```

কারণ **"2"** এবং **"1"** compare করলে **2** আগে বড়।

এটা beginnerদের জন্য একটি common trap।

43. Comparison Best Practice

যদি input string হয় কিন্তু তুমি number compare করতে চাও, explicit conversion করো।

Instead of relying on coercion:

```
if (input > 18) {  
  }  
}
```

এটা context অনুযায়ী কাজ করতে পারে, কিন্তু input-এর type জানা থাকলে আরও explicit:

```
const age = Number(input);  
  
if (age > 18) {  
  }  
}
```

এতে code intent clearer হয়।

44. Real-world Example — Age Check

```
const age = 25;  
  
if (age >= 18) {  
  console.log("Adult");  
} else {  
  console.log("Minor");  
}
```

এখানে:

```
age >= 18
```

একটি comparison expression।

Result:

```
true / false
```

45. Real-world Example — Login

```
const isLoggedIn = true;
const hasPermission = true;

if (isLoggedIn && hasPermission) {
  console.log("Access granted");
}
```

এখানে:

- comparison/logical condition
- boolean
- `&&`

সব একসাথে কাজ করছে।

46. Real-world Example — Search

ধরো search input:

```
const searchTerm = "";
```

তুমি চাইলে:

```
if (searchTerm) {
  console.log("Searching...");
}
```

Empty string falsy হওয়ায় search হবে না।

User লিখলে:

```
const searchTerm = "javascript";
```

এটি truthy হওয়ায় condition execute হবে।

47. Real-world Example — API Data

ধরো API থেকে:

```
const user = null;
```

তুমি লিখতে পারো:

```
if (user) {  
  console.log(user.name);  
}
```

`user` null হওয়ায় block execute হবে না।

আর modern JavaScript-এ optional chaining-ও useful:

```
console.log(user?.name);
```

এটি `user` null/undefined হলে error না দিয়ে `undefined` দিতে পারে।

48. Common Mistakes

Mistake 1 — Empty Array is Falsy ভাবা



```
if ([]) {  
  // ভাবা হচ্ছে এখানে false  
}
```

কিন্তু `[]` truthy।

Mistake 2 — Empty Object is Falsy ভাবা

```
Boolean({});  
// true
```

Mistake 3 — || দিয়ে সব default করা

```
const count = 0;

const result = count || 10;
```

Result:

```
10
```

যদি 0 valid হয়, এটি bug তৈরি করতে পারে।

সেক্ষেত্রে:

```
const result = count ?? 10;
```

Mistake 4 — && এবং || শুধু Boolean return করে ভাবা

```
console.log("Hello" && "World");
// "World"
```

```
console.log("" || "Guest");
// "Guest"
```

এগুলো actual operand return করতে পারে।

Mistake 5 — String Comparison-এর rules না বোঝা

```
"2" > "10"
```

দুইটিই string হলে lexical comparison হতে পারে।

49. Best Practices

1. Boolean condition simple রাখো

Good:

```
if (isLoggedIn && hasPermission) {  
  }  
}
```

2. Fallback-এর intent বুঝে operator choose করো

Falsy fallback দরকার হলে:

```
value || defaultValue
```

শুধু `null` / `undefined` fallback দরকার হলে:

```
value ?? defaultValue
```

3. Complex logical expressions-এ parentheses ব্যবহার করো

Instead of:

```
a && b || c && d
```

প্রয়োজনে:

```
(a && b) || (c && d)
```

4. User input-এর type সম্পর্কে সচেতন থাকো

Form input সাধারণত string হতে পারে।

তাই numeric comparison/calculation-এর আগে:

```
const age = Number(input);
```



Exercises

Exercise 1 — Truthy/Falsy

নিচের প্রতিটির output predict করো:

```
Boolean(0);  
Boolean(1);  
Boolean("");  
Boolean("hello");  
Boolean(null);  
Boolean(undefined);  
Boolean([]);  
Boolean({});  
Boolean(" ");
```

Exercise 2 — &&

Output predict করো:

```
console.log(true && "Hello");  
console.log(false && "Hello");  
console.log("JavaScript" && "React");  
console.log("" && "React");
```

Exercise 3 — ||

```
console.log(true || "Hello");  
console.log(false || "Hello");  
console.log("" || "Guest");  
console.log("Shariar" || "Guest");
```

Exercise 4 —

```
console.log(!true);  
  console.log(!false);  
  console.log(!0);  
  console.log(!1);  
  console.log(!"");  
  console.log(!"hello");
```

Exercise 5 —

```
console.log (!!0);  
  console.log (!!1);  
  console.log (!! "");  
  console.log (!! "hello");  
  console.log (!! []);  
  console.log (!! {});
```



Output Prediction Questions

আগে নিজে solve করবো।

Question 1

```
console.log(Boolean("0"));
```

Question 2

```
console.log(Boolean(" "));
```

Question 3

```
console.log(Boolean([]));
```

Question 4

```
console.log(Boolean({}));
```

Question 5

```
console.log(0 && "Hello");
```

Question 6

```
console.log(10 && "Hello");
```

Question 7

```
console.log("" || "Guest");
```

Question 8

```
console.log("Shariar" || "Guest");
```

Question 9

```
console.log(null || "Default");
```

Question 10

```
console.log(0 || 100);
```

Question 11

```
console.log(0 ?? 100);
```

Question 12

```
console.log(false || "Hello");
```

Question 13

```
console.log(false ?? "Hello");
```

Question 14

```
console.log("2" > "10");
```

Question 15

```
console.log("2" > 10);
```

Question 16

```
console.log(5 < 10);  
console.log(5 >= 10);
```

Question 17 — একটু tricky

```
console.log(true || false && false);
```

Step-by-step precedence চিন্তা করো।

Question 18 — Short-Circuit

```
false && console.log("Hello");  
  
true || console.log("World");
```

কোনো কিছু print হবে কি?

Interview Questions

1. Truthy value কী?
2. Falsy values কী কী?
3. JavaScript-এর সব falsy value বলতে কী বোঝায়?
4. Empty string এবং space string-এর difference কী?
5. Empty array truthy কেন?
6. Empty object truthy কেন?
7. `Boolean(value)` কী করে?

8. `!!value` কীভাবে কাজ করে?
9. `&&` operator কী?
10. `||` operator কী?
11. `&&` এবং `||` কি সবসময় boolean return করে?
12. `&&` কীভাবে short-circuit করে?
13. `||` কীভাবে short-circuit করে?
14. Short-circuit evaluation কী?
15. `value || defaultValue` কখন useful?
16. `||` এবং `??` -এর মধ্যে পার্থক্য কী?
17. `0 || 100` এবং `0 ?? 100` -এর result আলাদা কেন?
18. Comparison operators কী কী?
19. String comparison কীভাবে কাজ করে?
20. `"2" > "10"` কেন unexpected result দিতে পারে?
21. Complex logical expressions-এ parentheses কেন useful?
22. React-এ `condition && <Component />` pattern কীভাবে কাজ করে?

Coding Challenges

Challenge 1 — User Access

তোমার কাছে:

```
const isLoggedIn = true;
const isVerified = true;
const isBanned = false;
```

এমন condition তৈরি করো যাতে:

- user logged in হতে হবে
- verified হতে হবে
- banned হওয়া যাবে না

Access পেলো:

```
"Access granted"
```

না হলে:

```
"Access denied"
```

Challenge 2 — Default Username

একটি function তৈরি করো:

```
getUserName(name)
```

যদি valid name দেওয়া হয়:

```
getUserName("Shariar");  
// "Shariar"
```

যদি empty string দেওয়া হয়:

```
getUserName("");  
// "Guest"
```

 ব্যবহার করে solve করো।

Challenge 3 — Preserve Zero

একটি function তৈরি করো:

```
getQuantity(quantity)
```

যেখানে  একটি valid value।

```
getQuantity(5);  
// 5  
  
getQuantity(0);  
// 0  
  
getQuantity(null);  
// 1
```

এখানে `||` ব্যবহার করলে সমস্যা হবে।

Hint: `??` নিয়ে চিন্তা করো।

Challenge 4 — Age Validator

একটি function:

```
checkAge (age)
```

Requirements:

- age `18` বা তার বেশি হলে `"Adult"`
- age `18` -এর কম হলে `"Minor"`
- invalid/missing value হলে `"Invalid age"`

Example:

```
checkAge (25) ;  
  // "Adult"  
  
checkAge (15) ;  
  // "Minor"  
  
checkAge (null) ;  
  // "Invalid age"
```



Mini Project — Login Access Checker

একটি ছোট authentication logic তৈরি করো।

Data:

```
const user = {  
  isLoggedIn: true,  
  isVerified: true,  
  isBanned: false,  
  role: "admin"  
};
```

Requirements:

Rule 1

Logged in না হলে:

```
"Please login"
```

Rule 2

Logged in কিন্তু verified না হলে:

```
"Please verify your account"
```

Rule 3

Banned হলে:

```
"Access denied"
```

Rule 4

সব ঠিক থাকলে এবং role `admin` হলে:

```
"Welcome Admin"
```

Rule 5

সব ঠিক থাকলে কিন্তু admin না হলে:

```
"Welcome User"
```

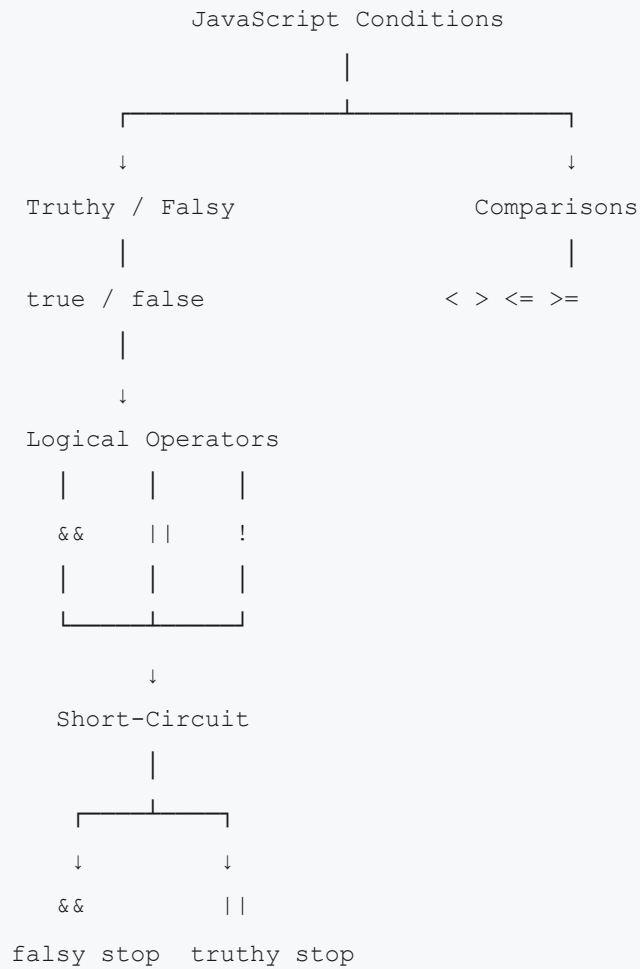
এখানে তোমাকে ব্যবহার করতে হবে:

- `if/else`
 - `comparison`
 - `&&`
 - `||` অথবা `??` যেখানে appropriate
 - `truthiness`
 - `boolean values`
-



Final Mental Model

এই chapter-এর পুরো logic:



আর fallback-এর ক্ষেত্রে:

`value || default`

↓

যেকোনো falsy হলে default

`value ?? default`

↓

শুধু null/undefined হলে default

Quick Revision

Truthy

```
Boolean("hello"); // true
Boolean(1);        // true
Boolean([]);       // true
Boolean({});       // true
```

Falsy

```
false
0
-0
0n
""
null
undefined
NaN
```

AND

```
A && B
```

A falsy হলে A, otherwise B।

OR

```
A || B
```

A truthy হলে A, otherwise B।

NOT

```
!value
```

Truthiness reverse করে।

Double NOT

```
!!value
```

Boolean result দেয়।

Nullish Coalescing

```
value ?? defaultValue
```

শুধু `null` এবং `undefined` হলে *fallback*।

Comparison

```
<  
>  
<=  
>=
```

সাধারণত `true` বা `false` return করে।



Golden Rules

1. `[]` এবং `{}` *truthy* — *empty* হলেও।

2. `&&` এবং `||` *actual operand value return* করতে পারে; শুধু `true/false` নয়।

3. `&&` *falsy value* পেলে *short-circuit* করে।

4. `||` *truthy value* পেলে *short-circuit* করে।

5. `||` এবং `??` এক জিনিস নয়।

6. *Numeric input* এবং *string input*-এর *comparison behavior* বুঝে কাজ করে।

7. *Complex logical expression* হলে *parentheses* ব্যবহার করে *intent clear* করে।